

ModelMatch — Phase 1 Build Prompt

Tagline: Find the Perfect AI for Every Project.

Read this first (context for the AI IDE)

ModelMatch is an AI recommendation engine: users describe a project in plain language, and the system recommends the best AI tools for it, with reasoning, comparisons, and a suggested workflow.

This document specifies **Phase 1 only** — a working end-to-end product with a real recommendation engine, not a static comparison site and not a UI mockup with fake data. Do not build authentication, admin panels, PWA support, or any "Future Features" listed at the bottom. Those are intentionally deferred so Phase 1 ships as something real and usable. The architecture should not actively block adding them later, but don't spend time on them now.

If anything in this spec is ambiguous, prefer the simplest version that still works end-to-end over a more elaborate version that's incomplete.

Tech Stack (Phase 1)

- Next.js (App Router) + React + TypeScript
 - Tailwind CSS + Shadcn UI
 - Framer Motion (use sparingly – entrance transitions and the workflow diagram, not everywhere)
 - An LLM API call (Claude or GPT) for the recommendation engine – see below
 - No database yet. Tool data lives in a local JSON file. No auth yet.
 - Deployable to Vercel
-

The Recommendation Engine (the core of Phase 1 – build this first)

This is the hardest and most important part. Build and test this before investing in UI polish.

How it works:

1. User submits a free-text description of their project (e.g. "I want to build a mobile app for tracking workouts, I don't know how to code, low budget").
2. The backend sends the user's text + the full tool dataset (see below) to an LLM in a single

structured prompt.

3. The LLM is instructed to:

- Extract the implicit requirements (project type, skill level, budget, need for coding/design/research/etc.)
- Select the best 3–6 tools from the provided dataset only (never invent tools or facts not in the dataset)
- Score each selected tool 0–100 for fit to *this specific request*
- Give a one-sentence reason for each, plus a short pros/cons
- Optionally suggest a multi-tool workflow (e.g. Research → Tool A, Coding → Tool B) if the project spans multiple needs

4. The LLM responds in **strict JSON only** (no prose, no markdown fences) matching a defined schema, so the frontend can render it directly.

5. Frontend renders this as comparison cards/table + an optional workflow diagram.

Grounding rule: the LLM must only recommend tools that exist in the provided dataset, and must base scores/reasoning on the data given, not on outside knowledge of the tools (which may be outdated or wrong). This keeps the output trustworthy and consistent.

Bias-control rule (important): whichever LLM provider you use to *run* the recommendation engine (e.g. Claude API, OpenAI API) is a different thing from the AI tools being *recommended* in the dataset (which may include that same provider's products, e.g. Claude or GPT as candidate tools). LLMs have a documented tendency toward mild self-preference bias — favoring outputs associated with their own maker. To prevent this from skewing recommendations:

- **Anonymize tool identity before sending the scoring prompt.** Strip real names/brands from the dataset and replace with neutral IDs (`tool_01` , `tool_02` , ...) before constructing the prompt sent to the LLM. The model scores `tool_07` , not "Claude" or "ChatGPT."
- Only after the LLM returns scores keyed by anonymized ID, map those IDs back to the real tool names/logos/links in your own code (not the LLM's) for display.
- This way the judge model never sees which tool belongs to which company, so it has nothing to be biased toward.

This is a one-time step in how you build the prompt (anonymize → score → re-map), not an ongoing cost, and it's worth keeping even after Phase 1 — it's a

meaningful trust/credibility feature for the product,
not just a nice-to-have.

Suggested response schema:

```
{
  "extracted_requirements": {
    "project_type": "string",
    "skill_level": "beginner | intermediate
| advanced",
    "budget": "free | low | medium | high |
unspecified",
    "needs": ["coding", "ui_design",
"research", "..."]
  },
  "recommendations": [
    {
      "tool_id": "string (matches dataset
id)",
      "fit_score": 0-100,
      "reason": "one sentence",
      "pros": ["..."],
      "cons": ["..."]
    }
  ],
  "suggested_workflow": [
    { "stage": "Research", "tool_id": "..."}
  ],
  { "stage": "Coding", "tool_id": "..."}
]
}
```

Build the dataset before building the prompt. Create `/data/tools.json` with ~20 real AI tools (mix of coding, image, writing, research, design tools). Each entry needs: id, name, logo URL, category, free/paid, monthly price, context length, strengths, weaknesses, website link, last-updated date. Hand-write this data accurately — it's the foundation the whole product's credibility rests on. Wrong pricing or fake context-length numbers will undermine trust immediately.

UI/UX Design Workflow — using Google Stitch

Before writing frontend code, use **Google Stitch** (stitch.withgoogle.com) to design the two Phase 1 screens (landing page, results page). Stitch turns natural-language prompts into UI mockups and can export to Figma or as code.

How to use it well:

- Write specific prompts, not vague ones — mention the product, the target user, the exact sections needed, and the style reference (Linear/Stripe — clean type, generous whitespace, one accent color, minimal gradients). Vague prompts produce generic results.

- Generate the landing page and results page as separate, clearly-described prompts so each stays focused.
- Iterate with follow-up prompts to fix spacing, color, or layout issues rather than starting over.
- Export to **Figma** if you want to refine details by hand, or export **code** (HTML/CSS or React) to hand to your AI IDE as a structural reference.

Important — treat Stitch's output as a reference, not final code. Stitch doesn't know your tech stack (Tailwind + Shadcn UI) and its exported code is a starting structural point, not production-ready. The right workflow is:

1. Design the screens in Stitch.
2. Export to Figma (for visual reference) or as code (for structural reference — component layout, hierarchy).
3. Give that exported reference to your AI IDE along with this spec, and have it **rebuild the screens properly** using Next.js + Tailwind + Shadcn UI — matching Stitch's layout and visual direction, not copy-pasting Stitch's raw markup.

This keeps Stitch doing what it's good at (fast visual exploration so you're not starting from a blank page) while your actual app stays clean and consistent in the real stack.

Pages (Phase 1)

1. Landing page

- Hero with the tagline and a large project-description input (this is the primary CTA – get users typing immediately, don't bury it)
- Brief "how it works" section (3 steps)
- A few example prompts the user can click to try instantly (great for first-time users who don't know what to type)
- Clean footer

2. Results page

- Shows extracted requirements as a few small tags/chips ("Mobile app", "Beginner", "Low budget") so the user can see the system understood them
- Comparison cards for each recommended tool: logo, name, fit score, one-line reason, pros/cons, pricing, link out
- If a workflow was suggested, show it as a simple horizontal step diagram (stage → tool), not necessarily fully interactive yet – a clean static visual is fine for Phase 1

- A "copy optimized prompt" button per tool (see below)

Prompt generator (lightweight for Phase 1)

For each recommended tool, generate one ready-to-use prompt the user could paste into that tool, tailored to their project description. This can be a second small LLM call or appended to the same response — simplest implementation wins. One-click copy to clipboard.

Design direction

Reference Linear and Stripe specifically — clean type hierarchy, generous whitespace, restrained color (one accent color, mostly neutral grays), subtle motion on state changes rather than constant animation. Avoid: purple-blue gradient backgrounds, glassmorphism by default, generic AI-startup look. Dark mode can come in Phase 2 if it adds friction now.

This is the same direction to use when prompting Stitch in the design step above — keep the two consistent so the AI IDE isn't reconciling two different visual languages.

Explicitly deferred to later phases (do not build now)

Authentication, user dashboard, saved searches/favorites, admin panel, PWA, browser/VS Code extensions, mobile apps, team collaboration, public API, enterprise plans, community reviews, prompt marketplace, benchmark-data ingestion, dynamic score updates from user feedback.

When these are eventually built, the scoring system should be able to evolve from "LLM reasons over a static JSON file" toward incorporating real benchmark data and user feedback signals — keep the tool dataset as a clean, swappable JSON/DB shape so that transition isn't a rewrite.

Definition of done for Phase 1

A user can land on the page, describe a real project in their own words, and within a few seconds get back accurate, well-reasoned tool recommendations grounded in real data — with no fake tools, no made-up stats, and no dead-end UI.